AdaptiveOps — Automated E-Commerce Deployment Platform

1. Executive summary

E-commerce service disruption and complex releases motivate an automated delivery system. The team will reuse the Google Online Boutique open-source application and implement the DevOps layer: Git-based teamwork, Jenkins pipelines, containerization, Kubernetes deployment and autoscaling, Nginx routing, Prometheus/Grafana monitoring, Ansible configuration, Terraform provisioning where feasible, and verifiable automated rollback.

The objective is a reproducible local demonstration. AWS deployment is limited to resources permitted by available access, cost constraints and the approved feasible scope; Terraform scripts remain part of the planned deliverables. Application deployment, tests, dashboards and cloud resources will be reported as complete only after verification.

2. Project scope

Included / planned
Pinned Online Boutique source, attribution, web/cart/mock payment and SearchProducts API verification
Docker image builds, GitHub workflow, Jenkins CI/CD, deployment smoke checks
Kubernetes deployment, HPA with valid metrics, Nginx access, Prometheus/Grafana views
Ansible scripts for real target setup, AWS Terraform for allowed/feasible subset
Runbooks, test evidence, contribution log

3. Internal schedule / Gantt overview

Workstream	16 Oct	17-23 Oct	24-30 Oct	31 Oct-6 Nov	7-13 Nov	14-20 Nov	21-27 Nov	28-30 Nov	1-4 Dec
October documentation	X								
App feasibility / Docker		X	X						
Jenkins CI / CD			X	X	X	X			
Kubernetes / Nginx			X	X	X				
Monitoring and HPA				X	X	X			
Ansible / AWS feasibility		X	X		X	X			
Rollback / integration					X	X	X	X	
Testing and final delivery							X	X	X

Official deadlines: 16 October (planning/review/requirements); 6 November (system design); 30 November (implementation); 4 December (final testing, reports and presentation).

4. Team roles and resource allocation

Person	Assigned primary responsibility	Hands-on cross-training
Amer Hossam Eldin Mohamed Mustafa	Leader; Jenkins CI/CD; integration; rollback	Docker, Kubernetes, monitoring
Kirollos Magdy Zekry Awad	Kubernetes; Nginx; HPA	Jenkins and Ansible
Rana Ashraf Abu Elyazeed Abu Elyazeed	Ansible; AWS/Terraform	Kubernetes and manifests
Rahma Mohamed Abdo El-Sharqawy	Prometheus; Grafana; KPIs	Docker, HPA and testing
Jana Mohamed Mahmoud El-Waleely	Online Boutique feasibility; Docker builds	CI and Kubernetes
Abdelrahman Ahmed Abdelbaset Mohamed	Smoke/load/rollback tests; execution docs	Jenkins and cluster ops

5. Risk assessment and mitigation

Identified risk	Mitigation and fallback
Ubuntu VM insufficient RAM / CPU	Measure guest resources; reduce footprint; split tools across VMs; verify early
Application dependency fails locally	Pin source; test SearchProducts API and Redis behavior; clarify UI search and durability expectations
HPA metrics unavailable	Install Metrics Server; set resource requests; verify scaling trigger with actual load
Rollback is only manual	Define rollout timeout and functional failures; retain known-good image/config; roll back and rerun checks

6. Proposed KPIs and measurement plan

KPI	Proposed target / acceptance test	Source of evidence
Reproducible deployment	Two clean deployment runs using documented steps	Jenkins logs; cluster state
CI quality gates	At least one automated test stage, image build and controlled deployment	Jenkins stage logs; Git commits
HPA scaling	Observe at least one eligible workload scale from 1 to >=2 replicas and return toward minimum	kubectl get hpa; metrics; load logs
Monitoring visibility	Grafana shows live service/infrastructure metrics from Prometheus	Target status; dashboard capture
Automated recovery	One intentionally faulty release triggers rollback without manual revision selection and passes recovery smoke test	Jenkins logs; Kubernetes rollout; screenshots
Configuration automation	One Ansible playbook configures a host; repeat execution does not introduce unintended changes	Ansible playbook output